Project II: Sliding Tiles

v1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Directory Hierarchy

1.1 Directories

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

Chapter 5

Directory Documentation

5.1 Project II Slider Game Directory Reference

Files

- file Board.cpp

  File containing the initialization of all Board methods.
- file Board.hpp

  File containing a Board class.
- file Main.cpp

  Main file where the program is executed.
- file SlidingTilesData.cpp

  File containing sliding tiles variable initializations.
- file SlidingTilesData.hpp

  File containing a namespace data for a Sliding Tiles game.
- file SlidingTilesEnums.hpp

  File containing a namespace with enum classes for a Sliding Tiles game.
- file SlidingTilesFunctions.cpp

  File containing the initialization of the SlidingTilesFunctions functions.
- file SlidingTilesFunctions.hpp

  File containing a namespace with declared functions used during a game of Sliding Tiles.
- file SlidingTilesFunctionsHelpers.cpp

  File containing the initialization of the SlidingTilesFunctionsHelpers functions.
- file SlidingTilesFunctionsHelpers.hpp

  File containing a namespace of declared helper functions and a constant expression for SlidingTilesFunctions.
- file UserFunctions.cpp

  File containing the initialization of the UserFunctions functions.
- file UserFunctions.hpp

  File containing a namespace with declared functions that the user interacts with.
- file UserFunctionsHelpers.cpp

  File containing the initialization of the UserFunctionsHelpers functions.
- file UserFunctionsHelpers.hpp

  File containing a namespace of declared helper functions and a constant expression for UserFunctions.

Chapter 6

Namespace Documentation

6.1 SlidingTilesData Namespace Reference

Namespace containing data for a Sliding Tiles game.

Variables

- Board board

  The `Board` object for Sliding Tiles.
- unsigned int slides = 0

  The amount of slides performed.
- size_t currentRow = 0

  The current row index of the empty tile.
- size_t currentColumn = 0

  The current column index of the empty tile.
- size_t boardSize = 0

  The size of the board in a single direction.
- SlidingTilesEnums::Difficulty currentDifficulty = SlidingTilesEnums::Difficulty::EASY

  The current difficulty being played.

6.1.1 Detailed Description

Namespace containing data that changes and persists between functions during a game of Sliding Tiles.

Author

Rajdeep Chowdhury

Date

8.04.2026

6.1.2 Variable Documentation

6.1.2.1 board

```
Board SlidingTilesData::board
```

The Board object which is used during a Sliding Tiles game.

Definition at line 20 of file SlidingTilesData.cpp.

6.1.2.2 boardSize

```
size_t SlidingTilesData::boardSize = 0
```

The amount of tiles the board has in a single direction.

Definition at line 23 of file SlidingTilesData.cpp.

6.1.2.3 currentColumn

```
size_t SlidingTilesData::currentColumn = 0
```

The column index of the empty tile of the Sliding Tiles game.

Definition at line 22 of file SlidingTilesData.cpp.

6.1.2.4 currentDifficulty

```
SlidingTilesEnums::Difficulty SlidingTilesData::currentDifficulty = SlidingTilesEnums::Difficulty::EASY
```

The difficulty of the Sliding Tiles game being played.

Definition at line 24 of file SlidingTilesData.cpp.

6.1.2.5 currentRow

```
size_t SlidingTilesData::currentRow = 0
```

The row index of the empty tile of the Sliding Tiles game.

Definition at line 22 of file SlidingTilesData.cpp.

6.1.2.6 slides

```
unsigned int SlidingTilesData::slides = 0
```

The amount of slides which occur during a Sliding Tiles game.

Definition at line 21 of file SlidingTilesData.cpp.

6.2 SlidingTilesEnums Namespace Reference

Namespace containing enum classes for a Sliding Tiles game.

Enumerations

- enum class Difficulty { EASY = 3 , MEDIUM = 4 , HARD = 6 , INSANE = 9 }

  Enum class containing difficulties for a Sliding Tiles game.
- enum class Direction { UP , DOWN , LEFT , RIGHT }

  Enum class containing directions that a slide can occur.

6.2.1 Detailed Description

Namespace containing enum classes that represent the difficulty of a Sliiding Tiles game and the directions which a slide can occur.

Author

Rajdeep Chowdhury

Date

8.04.2026

6.2.2 Enumeration Type Documentation

6.2.2.1 Difficulty

```
enum class SlidingTilesEnums::Difficulty  [strong]
```

Enum class containing difficulties for a Sliding Tiles game. The integers of each enum represent the size of board they produce.

Author

Rajdeep Chowdhury

Date

8.04.2026

Enumerator

EASY	EASY Difficulty. Has a board size of 3. The easy difficulty for Sliding Tiles. Has a board size of 3, meaning a 3x3 board is produced.

MEDIUM	MEDIUM Difficulty. Has a board size of 4. The medium difficulty for Sliding Tiles. Has a board size of 4, meaning a 4x4 board is produced.
HARD	HARD Difficulty. Has a board size of 6. The hard difficulty for Sliding Tiles. Has a board size of 6, meaning a 6x6 board is produced.
INSANE	INSANE Difficulty. Has a board size of 9. The insane difficulty for Sliding Tiles. Has a board size of 9, meaning a 9x9 board is produced.

Definition at line 24 of file SlidingTilesEnums.hpp.

6.2.2.2 Direction

```
enum class SlidingTilesEnums::Direction  [strong]
```

Enum class containing the four directions that a slide can be performed in. It is used in the context of the empty tile sliding to an adjacent filled tile, rather that the vice versa.

Author

Rajdeep Chowdhury

Date

8.04.2026

Enumerator

UP	UP Direction. Enum used to slide the empty tile up.
DOWN	DOWN Direction. Enum used to slide the empty tile down.
LEFT	LEFT Direction. Enum used to slide the empty tile left.
RIGHT	RIGHT Direction. Enum used to slide the empty tile right.

Definition at line 51 of file SlidingTilesEnums.hpp.

6.3 SlidingTilesFunctions Namespace Reference

Namespace containing the functions used during a game of Sliding Tiles.

Functions

- void startGame (SlidingTilesEnums::Difficulty difficulty)

  Sets up the data required to start a Sliding Tiles game.
- bool slide (SlidingTilesEnums::Direction direction)

  Attempts to slide the empty tile to an adjacent filled tile.
- void shuffle ()

  Shuffles the board.
- bool isBoardOrdered ()

  Checks if all the board's numbers are ordered.
- void writeScore (SlidingTilesEnums::Difficulty difficulty)

  Writes the number of slides to its associated leaderboard.
- std::vector< unsigned int > loadScores (SlidingTilesEnums::Difficulty difficulty)

  Loads the top scores from its associated leaderboard.

6.3.1 Detailed Description

Namespace containing the functions which are called during a game of Sliding Tiles to change the properties of SlidingTilesData.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

SlidingTilesData

6.3.2 Function Documentation

6.3.2.1 isBoardOrdered()

```
bool SlidingTilesFunctions::isBoardOrdered ()
```

Checks if all the board's numbers are in their expected, ordered position, reading the board from left to right and from top to bottom.

Return values

true	If the board is ordered.
false	If the board is not ordered.

Author

Fabio Bustamante Romillo

Date

8.04.2026

Definition at line 114 of file SlidingTilesFunctions.cpp.

6.3.2.2 loadScores()

```
std::vector< unsigned int > SlidingTilesFunctions::loadScores (
            SlidingTilesEnums::Difficulty difficulty)
```

Loads all scores from a text file representing the leaderboard of the given `difficulty` into a vector, sorts them in ascneding order, then filters the scores to the top SlidingTilesFunctionsHelpers::LEADERBOARD_SIZE scores.

Parameters

`in`	*difficulty*	The difficulty of the Sliding Tiles game leaderboard.

Returns

A vector of the top scores.

Author

Raj Bahadur Bhat

Date

8.04.2026

See also

SlidingTilesFunctionsHelpers::LEADERBOARD_SIZE

Definition at line 155 of file SlidingTilesFunctions.cpp.

6.3.2.3 shuffle()

```
void SlidingTilesFunctions::shuffle ()
```

Shuffles SlidingTilesData::board by repeatedly calling slide(SlidingTilesEnums::Direction) with a random direction. The shuffle will not immediately backtrack, meaning it can't slide one direction and then slide back the opposite direction immediately afterwards.

Precondition

SlidingTilesData::board is not empty and all rows contain at least one element. (i.e., the Board::hasEmptyRow() call returns false.)

Author

Fabio Bustamante Romillo

Date

8.04.2026

Note

If the pre-condition is not met, the function returns early without modifying the data.

The function SlidingTilesFunctions::StartGame(SlidingTilesEnums::Difficulty) should be used before calling this function.

Warning

If either one of the dimensions of the SlidingTilesData::board is somehow equal to 1, this function will enter an infinite loop due to its incapability to backtrack. This should be impossible at the moment, but it is worth noting if this code is to be expanded on.

See also

SlidingTilesFunctionsHelpers

Board::hasEmptyRow()

Definition at line 87 of file SlidingTilesFunctions.cpp.

6.3.2.4 slide()

```
bool SlidingTilesFunctions::slide (
            SlidingTilesEnums::Direction direction)
```

Attempts to swap the empty tile with an adjacent filled tile based on the `direction` given. A slide will fail if the direction of slide given would cause the empty tile to slide outside the board.

Precondition

SlidingTilesData::board is not empty and all rows contain at least one element. (i.e., the Board::hasEmptyRow() call returns false.)

Parameters

`in`	*direction*	The direction that the slide will occur.

Return values

true	If the slide succeeds.
false	If the slide fails.
false	If the pre-condition is not met.

Author

Rajdeep Chowdhury

Date

8.04.2026

Note

If the pre-condition is not met, the function returns early without attempting to modify the data.

This function does not increment SlidingTilesData::slides. The caller is recommended to increment it based on if this function is true.

Definition at line 41 of file SlidingTilesFunctions.cpp.

6.3.2.5 startGame()

```
void SlidingTilesFunctions::startGame (
            SlidingTilesEnums::Difficulty difficulty)
```

Reintializes the data in SlidingTilesData to a reset state based on the `difficulty`, allowing for a new game of Sliding Tiles to be played.

Parameters

`in`	*difficulty*	The difficulty of the Sliding Tiles game being started.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

SlidingTilesData

Definition at line 31 of file SlidingTilesFunctions.cpp.

6.3.2.6 writeScore()

```
void SlidingTilesFunctions::writeScore (
            SlidingTilesEnums::Difficulty difficulty)
```

Appends the current number of SlidingTilesData::slides to the end of a text file representing the leaderboard of the given `difficulty`.

Parameters

`in`	*difficulty*	The difficulty of the Sliding Tiles game being played.

Author

Raj Bahadur Bhat

Date

8.04.2026

Note

The amount of scores that can be in a leaderboard text file has no limit.

Definition at line 129 of file SlidingTilesFunctions.cpp.

6.4 SlidingTilesFunctionsHelpers Namespace Reference

Namespace containing helper functions and a constant expression used by `SlidingTilesFunctions`.

Functions

- SlidingTilesEnums::Direction getRandomDirection ()

  Generates a random direction.
- SlidingTilesEnums::Direction getOpposite (SlidingTilesEnums::Direction dir)

  Converts the direction given into its opposite direction.

Variables

- constexpr unsigned short LEADERBOARD_SIZE = 10

  Constant representing the size of the leaderboard.

6.4.1 Detailed Description

Namespace of helper functions and a `constexpr` integer that the functions of SlidingTilesFunctions use.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

SlidingTilesFunctions

6.4.2 Function Documentation

6.4.2.1 getOpposite()

```
SlidingTilesEnums::Direction SlidingTilesFunctionsHelpers::getOpposite (
            SlidingTilesEnums::Direction dir)
```

Converts `dir` into the SlidingTilesEnums::Direction representation of the logical/common-sense opposite direction.

Parameters

`in`	*dir*	The direction which will be converted.

Return values

SlidingTilesEnums::Direction::UP	If the direction given is `SlidingTilesEnums::Direction::DOWN`
SlidingTilesEnums::Direction::DOWN	If the direction given is `SlidingTilesEnums::Direction::UP`
SlidingTilesEnums::Direction::LEFT	If the direction given is `SlidingTilesEnums::Direction::RIGHT`
SlidingTilesEnums::Direction::RIGHT	If the direction given is `SlidingTilesEnums::Direction::LEFT`

Author

Fabio Bustamante Romillo

Date

8.04.2026

Definition at line 26 of file SlidingTilesFunctionsHelpers.cpp.

6.4.2.2 getRandomDirection()

```
SlidingTilesEnums::Direction SlidingTilesFunctionsHelpers::getRandomDirection ()
```

Generates a random SlidingTilesEnums::Direction

Return values

SlidingTilesEnums::Direction::UP	1/4 chance.
SlidingTilesEnums::Direction::DOWN	1/4 chance.
SlidingTilesEnums::Direction::LEFT	1/4 chance.
SlidingTilesEnums::Direction::RIGHT	1/4 chance.

Author

Fabio Bustamante Romillo

Date

8.04.2026

Definition at line 21 of file SlidingTilesFunctionsHelpers.cpp.

6.4.3 Variable Documentation

6.4.3.1 LEADERBOARD_SIZE

```
unsigned short SlidingTilesFunctionsHelpers::LEADERBOARD_SIZE = 10  [constexpr]
```

Constant integer representing how many scores are loaded by the leaderboard.

Definition at line 32 of file SlidingTilesFunctionsHelpers.hpp.

6.5 UserFunctions Namespace Reference

Namespace containing functions that the user interacts with.

Functions

- bool selectDifficulty ()

  Allows the user to select their Sliding Tiles game difficulty.
- bool performSlide ()

  Allows the user to perform a slide in a Sliding Tiles game.

6.5.1 Detailed Description

Namespace containing functions that turn user input into output. It is the module that users directly interact with.

Author

Rajdeep Chowdhury

Date

8.04.2026

6.5.2 Function Documentation

6.5.2.1 performSlide()

```
bool UserFunctions::performSlide ()
```

Prompts the user to slide the empty tile in all SlidingTilesEnums::Direction available or to quit the game they are currently in. Sliding will affect the game state and end the game if the win condition is met.

Return values

true	If quit is not selected or the game is not over.
false	If quit is selected
false	If the game determines it can end.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 67 of file UserFunctions.cpp.

6.5.2.2 selectDifficulty()

```
bool UserFunctions::selectDifficulty ()
```

Prompts the user to select their SlidingTilesEnums::Difficulty or to quit. Selecting a difficulty will begin a game afterwards. Quitting will end program execution.

Return values

true	If quit is not selected.
false	If quit is selected.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 29 of file UserFunctions.cpp.

6.6 UserFunctionsHelpers Namespace Reference

Namespace containing helper functions used by `UserFunctions`.

Functions

- void startGame ()
  Initializes data to start a game of Sliding Tiles.
- void trySlide (SlidingTilesEnums::Direction direction)
  Tries to slide the empty tile in the given direction.
- void printBoard ()
  Print the Sliding Tiles board.
- void printLeaderboard ()
  Print the Sliding Tiles leaderboard for the current difficulty.
- bool continueGame ()
  Determines whether the Sliding Tiles game should continue.

6.6.1 Detailed Description

Namespace containing helper functions that the functions of UserFunctions use.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

UserFunctions

6.6.2 Function Documentation

6.6.2.1 continueGame()

```
bool UserFunctionsHelpers::continueGame ()
```

Determines whether the Sliding Tiles game should continue based on if the board is ordered or not. If the board is ordered, it will also handle applying the effect of winning the game.

Return values

true	If the game must still continue.
false	If the game should end.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 62 of file UserFunctionsHelpers.cpp.

6.6.2.2 printBoard()

```
void UserFunctionsHelpers::printBoard ()
```

Displays all the tiles of the Sliding Tiles board onto the console.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 39 of file UserFunctionsHelpers.cpp.

6.6.2.3 printLeaderboard()

```
void UserFunctionsHelpers::printLeaderboard ()
```

Displays the top SlidingTilesFunctionsHelpers::LEADERBOARD_SIZE scores achieved for the SlidingTilesData::currentDifficulty being played.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

SlidingTilesFunctionsHelpers
SlidingTilesData

Definition at line 52 of file UserFunctionsHelpers.cpp.

6.6.2.4 startGame()

```
void UserFunctionsHelpers::startGame ()
```

Uses functions from SlidingTilesFunctions to initialize the data required to start a game of Sliding Tiles.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 26 of file UserFunctionsHelpers.cpp.

6.6.2.5 trySlide()

```
void UserFunctionsHelpers::trySlide (
            SlidingTilesEnums::Direction direction)
```

Tries to slide the empty tile in the given directionm handling both the success and failure cases.

Parameters

in	*direction*	The direction the slide will occur.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 32 of file UserFunctionsHelpers.cpp.

Chapter 7

Class Documentation

7.1 Board Class Reference

A class that represents a board.

`#include <Board.hpp>`

Public Member Functions

- Board () noexcept

  `Board` object constructor.
- Board (SlidingTilesEnums::Difficulty difficulty) noexcept

  `Board` object constructor.
- bool hasEmptyRow () const noexcept

  Checks if the board has an empty row or is empty.
- bool canAccess (size_t row, size_t column) const noexcept

  Checks if a `Board` index can be accessed. Index starts at 0.
- bool canAccess (size_t row) const noexcept

  Checks if a `Board` row can be accessed. Index starts at 0.
- size_t & access (size_t row, size_t column)

  Accesses an integer in the board. Indexing starts at 0.
- std::vector< size_t > & access (size_t row)

  Accesses a row in the board. Indexing starts at 0.

Protected Attributes

- std::vector< std::vector< size_t > > board {}

  The data representation of the board.

7.1.1 Detailed Description

A class that represents a board for any game that can be represented by numbered tiles.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 26 of file Board.hpp.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 Board() [1/2]

```
Board::Board ()  [noexcept]
```

The default constructor for the `Board` that creates an empty, sizeless board.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 18 of file Board.cpp.

7.1.2.2 Board() [2/2]

```
Board::Board (
            SlidingTilesEnums::Difficulty difficulty)  [noexcept]
```

The parameterized constructor for the `Board` that creates a board based on the difficulty passed through for Sliding Tiles.

Parameters

in	*difficulty*	The difficulty of the Sliding Tiles game desired.

Author

Raj Bahadur Bhat

Date

8.04.2026

See also

SlidingTilesEnums

Definition at line 20 of file Board.cpp.

7.1.3 Member Function Documentation

7.1.3.1 access() [1/2]

```
std::vector< size_t > & Board::access (
            size_t row)
```

Returns a reference to the size_t row vector at the specified `row` index.

Parameters

in	*row*	The row of the board being accessed

Returns

A reference to the row vector on the board.

Exceptions

std::out_of_range	If the row being accessed does not exist.

Author

Rajdeep Chowdhury

Date

8.04.2026

Note

The user should use the canAccess(size_t) method before using this method to avoid the exception being thrown.

See also

canAccess(size_t)

Definition at line 44 of file Board.cpp.

7.1.3.2 access() [2/2]

```
size_t & Board::access (
            size_t row,
            size_t column)
```

Returns a reference to the size_t integer at the specified `row` and `column` index.

Parameters

in	*row*	The row of the board being accessed
in	*column*	The column of the board being accessed.

Returns

A reference to the size_t integer on the board.

Exceptions

std::out_of_range	If the row or column being accessed does not exist.

Author

Rajdeep Chowdhury

Date

8.04.2026

Note

The user should use the canAccess(size_t, size_t) method before using this method to avoid the exception being thrown.

See also

canAccess(size_t, size_t)

Definition at line 48 of file Board.cpp.

7.1.3.3 canAccess() [1/2]

```
bool Board::canAccess (
            size_t row) const  [noexcept]
```

Checks if a Board row can exist in accordance to the size of the vector representing the board rows. Index starts at 0.

Parameters

in	*row*	The row of the board being accessed.

Return values

true	If the row exists.
false	If the row does not exist.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 52 of file Board.cpp.

7.1.3.4 canAccess() [2/2]

```
bool Board::canAccess (
            size_t row,
            size_t column) const  [noexcept]
```

Checks if a Board index can exist in accordance to the size of the vectors representing the board. Index starts at 0.

Parameters

in	*row*	The row of the board being accessed.
in	*column*	The coloum of the board being accessed.

Return values

true	If the index exists.
false	If the index does not exist.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 56 of file Board.cpp.

7.1.3.5 hasEmptyRow()

```
bool Board::hasEmptyRow () const  [noexcept]
```

Checks if one of the vector sizes of a board row is 0. Also checks for if the board itself has no rows.

Return values

true	If an empty row exists.
true	If the board itself is empty.
false	If neither condition is true.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 36 of file Board.cpp.

7.1.4 Member Data Documentation

7.1.4.1 board

```
std::vector<std::vector<size_t> > Board::board {}  [protected]
```

The vector of vectors of size_t which represents a board.

Definition at line 30 of file Board.hpp.

The documentation for this class was generated from the following files:

- Board.hpp
- Board.cpp

Chapter 8

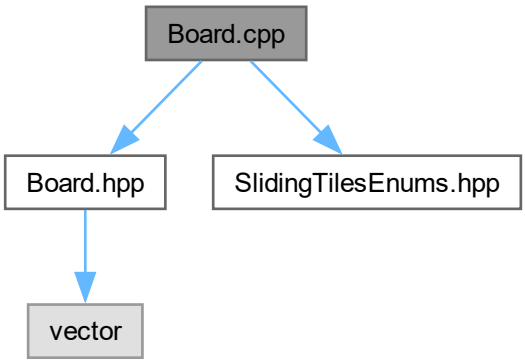
File Documentation

8.1 Board.cpp File Reference

File containing the initialization of all Board methods.

```
#include "Board.hpp"
#include "SlidingTilesEnums.hpp"
```

Include dependency graph for Board.cpp:



8.1.1 Detailed Description

This file contains the initialization of all Board methods declared in Board.hpp.

Author

Rajdeep Chowdhury

Raj Bahadur Bhat

Date

8.04.2026

See also

Board.hpp

SlidingTilesEnums.hpp

Definition in file Board.cpp.

8.2 Board.cpp

Go to the documentation of this file.

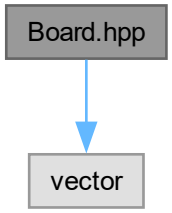
```
00001
00015 #include "Board.hpp"
00016 #include "SlidingTilesEnums.hpp"
00017
00018 Board::Board() noexcept {}
00019
00020 Board::Board(SlidingTilesEnums::Difficulty difficulty) noexcept {
00021     int size = static_cast <int> (difficulty); //Convert the enum into the board size
00022
00023     //Build the board
00024     board.resize(size);
00025     int counter = 1;
00026     for (int col = 0; col < size; col++) {
00027         board.at(col).resize(size);
00028
00029         for (int row = 0; row < size; row++) {
00030             board.at(col).at(row) = counter;
00031             counter++;
00032         }
00033     }
00034 }
00035
00036 bool Board::hasEmptyRow() const noexcept {
00037     if (this->board.size() <= 0) return true; //Check if board itself is empty
00038     for (int i = 0; i < board.size(); i++) { //Check each row for an empty row
00039         if (board.at(i).size() <= 0) return true;
00040     }
00041     return false;
00042 }
00043
00044 std::vector<size_t>& Board::access(size_t row) {
00045     return this->board.at(row);
00046 }
00047
00048 size_t& Board::access(size_t row, size_t column) {
00049     return this->access(row).at(column);
00050 }
00051
00052 bool Board::canAccess(size_t row) const noexcept {
00053     return (row >= 0 && row < this->board.size());
00054 }
00055
00056 bool Board::canAccess(size_t row, size_t column) const noexcept {
00057     if (!canAccess(row)) return false;
00058     size_t size = this->board.at(row).size();
00059     return (column >= 0 && column < size);
00060 }
```

8.3 Board.hpp File Reference

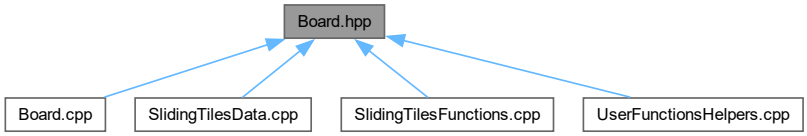
File containing a Board class.

```
#include <vector>
```

Include dependency graph for Board.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class Board

  A class that represents a board.

Namespaces

- namespace SlidingTilesEnums

  Namespace containing enum classes for a Sliding Tiles game.

8.3.1 Detailed Description

This file contains a class representing a board for any game, with the declaration of its method and attributes initialized.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

Board.cpp

Definition in file Board.hpp.

8.4 Board.hpp

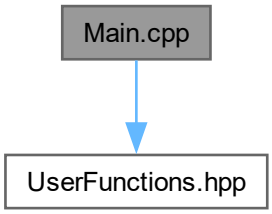
Go to the documentation of this file.

```
00001
00013 #pragma once
00014 #include <vector>
00015
00016 //Forward declaration
00017
00018 namespace SlidingTilesEnums {
00019     enum class Difficulty;
00020 }
00021
00026 class Board {
00027 protected:
00030     std::vector<std::vector<size_t» board{};
00031
00032 public:
00037     Board() noexcept;
00038
00047     Board(SlidingTilesEnums::Difficulty difficulty) noexcept;
00048
00057     bool hasEmptyRow() const noexcept;
00058
00068     bool canAccess(size_t row, size_t column) const noexcept;
00069
00078     bool canAccess(size_t row) const noexcept;
00079
00093     size_t& access(size_t row, size_t column);
00094
00107     std::vector<size_t>& access(size_t row);
00108 };
```

8.5 Main.cpp File Reference

Main file where the program is executed.

`#include "UserFunctions.hpp"`
Include dependency graph for Main.cpp:



Functions

- int main ()

  Execute the program.

8.5.1 Detailed Description

This file is the main file. It is where the functions that start the program are called.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

UserFunctions.hpp

Definition in file Main.cpp.

8.5.2 Function Documentation

8.5.2.1 main()

```
int main ()
```

The main function that executes the program.

Returns

Exit code.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition at line 19 of file Main.cpp.

8.6 Main.cpp

Go to the documentation of this file.

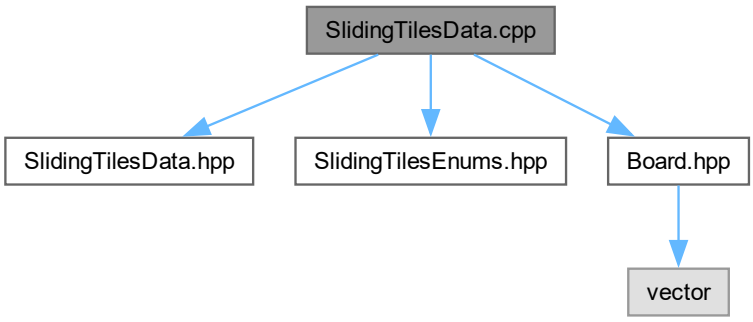
```
00001
00012 #include "UserFunctions.hpp"
00013
00019 int main() {
00020     while (UserFunctions::selectDifficulty()) {
00021         while (UserFunctions::performSlide());
00022     }
00023 }
```

8.7 SlidingTilesData.cpp File Reference

File containing sliding tiles variable initializations.

```
#include "SlidingTilesData.hpp"
#include "SlidingTilesEnums.hpp"
#include "Board.hpp"
```

Include dependency graph for SlidingTilesData.cpp:



Namespaces

- namespace SlidingTilesData

  Namespace containing data for a Sliding Tiles game.

Variables

- Board SlidingTilesData::board

  The `Board` object for Sliding Tiles.
- unsigned int SlidingTilesData::slides = 0

  The amount of slides performed.
- size_t SlidingTilesData::currentRow = 0

  The current row index of the empty tile.
- size_t SlidingTilesData::currentColumn = 0

  The current column index of the empty tile.
- size_t SlidingTilesData::boardSize = 0

  The size of the board in a single direction.
- SlidingTilesEnums::Difficulty SlidingTilesData::currentDifficulty = SlidingTilesEnums::Difficulty::EASY

  The current difficulty being played.

8.7.1 Detailed Description

This file contains the initializations of the externally linked SlidingTilesData namespace varaibles found in SlidingTilesData.hpp.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

SlidingTilesData.hpp

SlidingTilesEnums.hpp

Board.hpp

Definition in file SlidingTilesData.cpp.

8.8 SlidingTilesData.cpp

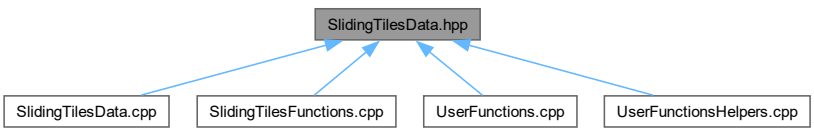
Go to the documentation of this file.

```
00001
00015 #include "SlidingTilesData.hpp"
00016 #include "SlidingTilesEnums.hpp"
00017 #include "Board.hpp"
00018
00019 namespace SlidingTilesData {
00020     Board board;
00021     unsigned int slides = 0;
00022     size_t currentRow = 0, currentColumn = 0;
00023     size_t boardSize = 0;
00024     SlidingTilesEnums::Difficulty currentDifficulty = SlidingTilesEnums::Difficulty::EASY;
00025 }
```

8.9 SlidingTilesData.hpp File Reference

File containing a namespace data for a Sliding Tiles game.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace SlidingTilesEnums

  Namespace containing enum classes for a Sliding Tiles game.
- namespace SlidingTilesData

  Namespace containing data for a Sliding Tiles game.

8.9.1 Detailed Description

This file contains a namespace with external links to data that change and persist during a Sliding Tiles game.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

SlidingTilesData.cpp

Definition in file SlidingTilesData.hpp.

8.10 SlidingTilesData.hpp

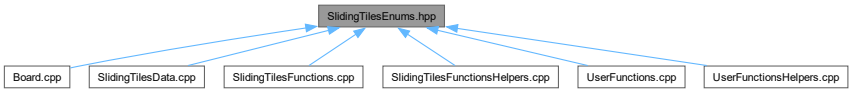
Go to the documentation of this file.

```
00001
00013 #pragma once
00014
00015 //Forward declarations
00016
00017 class Board;
00018
00019 namespace SlidingTilesEnums {
00020     enum class Difficulty;
00021 }
00022
00028 namespace SlidingTilesData {
00031     extern Board board;
00032
00035     extern unsigned int slides;
00036
00039     extern size_t currentRow;
00040
00043     extern size_t currentColumn;
00044
00047     extern size_t boardSize;
00048
00051     extern SlidingTilesEnums::Difficulty currentDifficulty;
00052 }
00053
```

8.11 SlidingTilesEnums.hpp File Reference

File containing a namespace with enum classes for a Sliding Tiles game.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace SlidingTilesEnums

  Namespace containing enum classes for a Sliding Tiles game.

Enumerations

- enum class SlidingTilesEnums::Difficulty { SlidingTilesEnums::EASY = 3 , SlidingTilesEnums::MEDIUM = 4 , SlidingTilesEnums::HARD = 6 , SlidingTilesEnums::INSANE = 9 }

  Enum class containing difficulties for a Sliding Tiles game.
- enum class SlidingTilesEnums::Direction { SlidingTilesEnums::UP , SlidingTilesEnums::DOWN , SlidingTilesEnums::LEFT , SlidingTilesEnums::RIGHT }

  Enum class containing directions that a slide can occur.

8.11.1 Detailed Description

This file contains a namespace with enum classes that represent the difficulty of a Sliding Tiles game and the directions which a slide can occur.

Author

Rajdeep Chowdhury

Date

8.04.2026

Definition in file SlidingTilesEnums.hpp.

8.12 SlidingTilesEnums.hpp

Go to the documentation of this file.

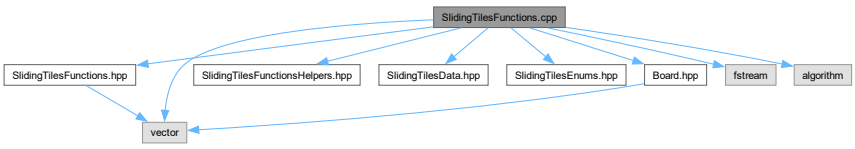
```
00001
00011 #pragma once
00012
00018 namespace SlidingTilesEnums {
00024     enum class Difficulty {
00028         EASY = 3,
00029
00033         MEDIUM = 4,
00034
00038         HARD = 6,
00039
00043         INSANE = 9
00044     };
00045
00051     enum class Direction {
00054         UP,
00055
00058         DOWN,
00059
00062         LEFT,
00063
00066         RIGHT
00067     };
00068 }
```

8.13 SlidingTilesFunctions.cpp File Reference

File containing the initialization of the SlidingTilesFunctions functions.

```
#include "SlidingTilesFunctions.hpp"
#include "SlidingTilesFunctionsHelpers.hpp"
#include "SlidingTilesData.hpp"
#include "SlidingTilesEnums.hpp"
#include "Board.hpp"
#include <vector>
#include <fstream>
#include <algorithm>
```

Include dependency graph for SlidingTilesFunctions.cpp:



Namespaces

- namespace SlidingTilesFunctions

  Namespace containing the functions used during a game of Sliding Tiles.

Functions

- void SlidingTilesFunctions::startGame (SlidingTilesEnums::Difficulty difficulty)

  Sets up the data required to start a Sliding Tiles game.
- bool SlidingTilesFunctions::slide (SlidingTilesEnums::Direction direction)

  Attempts to slide the empty tile to an adjacent filled tile.
- void SlidingTilesFunctions::shuffle ()

  Shuffles the board.
- bool SlidingTilesFunctions::isBoardOrdered ()

  Checks if all the board's numbers are ordered.
- void SlidingTilesFunctions::writeScore (SlidingTilesEnums::Difficulty difficulty)

  Writes the number of slides to its associated leaderboard.
- std::vector< unsigned int > SlidingTilesFunctions::loadScores (SlidingTilesEnums::Difficulty difficulty)

  Loads the top scores from its associated leaderboard.

8.13.1 Detailed Description

This file contains the initialization of the SlidingTilesFunctions functions declared in SlidingTilesFunctions.hpp.

Author

Rajdeep Chowdhury

Fabio Bustamante Romillo

Raj Bahadur Bhat

Date

9.04.2026

See also

SlidingTilesFunctions.hpp

SldingTilesFunctionsHelpers.hpp

SlidingTilesData.hpp

SlidingTilesEnums.hpp

Board.hpp

SlidingTilesData

Definition in file SlidingTilesFunctions.cpp.

8.14 SlidingTilesFunctions.cpp

Go to the documentation of this file.

```
00001
00020 #include "SlidingTilesFunctions.hpp"
00021 #include "SlidingTilesFunctionsHelpers.hpp"
00022 #include "SlidingTilesData.hpp"
00023 #include "SlidingTilesEnums.hpp"
00024 #include "Board.hpp"
00025 #include <vector>
00026 #include <fstream>
00027 #include <algorithm>
00028
00029 namespace SlidingTilesFunctions {
00030
00031     void startGame(SlidingTilesEnums::Difficulty difficulty) {
00032         SlidingTilesData::board = Board(difficulty);
00033         SlidingTilesData::boardSize = static_cast<int>(difficulty);
00034         SlidingTilesData::currentColumn =
00035             SlidingTilesData::currentRow =
00036             SlidingTilesData::boardSize - 1;
00037         SlidingTilesData::slides = 0;
00038         SlidingTilesData::currentDifficulty = difficulty;
00039     }
00040
00041     bool slide(SlidingTilesEnums::Direction direction) {
00042         using namespace SlidingTilesData;
00043         bool canSlide = false;
00044         //The main purpose of the below check is to ensure the board is not empty, meaning the default
      board is the current board.
00045         if (board.hasEmptyRow()) return false;
00046
00047         //Determine if a slide can occur in the desired direction
00048         switch (direction) {
00049         case SlidingTilesEnums::Direction::DOWN:
00050             canSlide = board.canAccess(currentRow + 1, currentColumn);
00051             break;
00052         case SlidingTilesEnums::Direction::UP:
00053             canSlide = board.canAccess(currentRow - 1, currentColumn);
00054             break;
00055         case SlidingTilesEnums::Direction::LEFT:
00056             canSlide = board.canAccess(currentRow, currentColumn - 1);
00057             break;
00058         case SlidingTilesEnums::Direction::RIGHT:
00059             canSlide = board.canAccess(currentRow, currentColumn + 1);
00060             break;
00061         }
00062
00063         if (!canSlide) return false;
00064
00065         //Slide the desired square to the empty square
00066         switch (direction) {
00067         case SlidingTilesEnums::Direction::DOWN:
00068             std::swap(board.access(currentRow, currentColumn), board.access(currentRow + 1,
      currentColumn));
00069             currentRow += 1;
00070             break;
00071         case SlidingTilesEnums::Direction::UP:
00072             std::swap(board.access(currentRow, currentColumn), board.access(currentRow - 1,
      currentColumn));
00073             currentRow -= 1;
00074             break;
00075         case SlidingTilesEnums::Direction::LEFT:
00076             std::swap(board.access(currentRow, currentColumn), board.access(currentRow, currentColumn
      - 1));
00077             currentColumn -= 1;
00078             break;
00079         case SlidingTilesEnums::Direction::RIGHT:
00080             std::swap(board.access(currentRow, currentColumn), board.access(currentRow, currentColumn
      + 1));
00081             currentColumn += 1;
00082             break;
00083         }
00084         return true;
00085     }
00086
00087     void shuffle() {
00088         std::srand(static_cast<unsigned int>(std::time(nullptr)));
00089
00090         size_t moves = SlidingTilesData::boardSize * SlidingTilesData::boardSize * 10;
00091
00092         SlidingTilesEnums::Direction lastMove = SlidingTilesEnums::Direction::UP;
00093         bool hasLastMove = false;
00094         int i = 0;
00095         while (i < moves) {
```

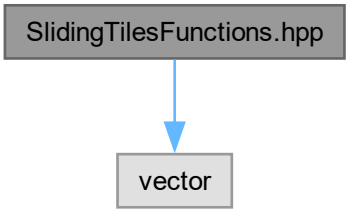
```
00096             SlidingTilesEnums::Direction move = SlidingTilesFunctionsHelpers::getRandomDirection();
00097
00098             // validMove = true only if move is not opposite and slide succeeds
00099             bool validMove = (!hasLastMove || move !=
    SlidingTilesFunctionsHelpers::getOpposite(lastMove)) && slide(move);
00100
00101             // increment counter if valid
00102             i += validMove;                          // true = 1, false = 0
00103             lastMove = validMove ? move : lastMove;  // update lastMove only if valid
00104             hasLastMove = hasLastMove || validMove;  // mark hasLastMove if any move succeeded
00105         }
00106
00107         // Move empty tile to bottom right
00108         while (SlidingTilesData::currentRow < SlidingTilesData::boardSize - 1)
00109             slide(SlidingTilesEnums::Direction::DOWN);
00110         while (SlidingTilesData::currentColumn < SlidingTilesData::boardSize - 1)
00111             slide(SlidingTilesEnums::Direction::RIGHT);
00112     }
00113
00114     bool isBoardOrdered() {
00115         bool isOrdered = true;
00116
00117         for (size_t row = 0; row < SlidingTilesData::boardSize; row++) {
00118             for (size_t col = 0; col < SlidingTilesData::boardSize; col++) {
00119
00120                 size_t expectedValue = row * SlidingTilesData::boardSize + col + 1;
00121
00122                 isOrdered = isOrdered && (SlidingTilesData::board.access(row, col) == expectedValue);
00123             }
00124         }
00125
00126         return isOrdered;
00127     }
00128
00129     void writeScore(SlidingTilesEnums::Difficulty difficulty) {
00130         // Get filename based on difficulty
00131         std::string filename;
00132         switch (difficulty) {
00133         case SlidingTilesEnums::Difficulty::EASY:
00134             filename = "scores_easy.txt";
00135             break;
00136         case SlidingTilesEnums::Difficulty::MEDIUM:
00137             filename = "scores_medium.txt";
00138             break;
00139         case SlidingTilesEnums::Difficulty::HARD:
00140             filename = "scores_hard.txt";
00141             break;
00142         case SlidingTilesEnums::Difficulty::INSANE:
00143             filename = "scores_insane.txt";
00144             break;
00145         }
00146
00147         // Open file to append score
00148         std::ofstream outFile(filename, std::ios::app);
00149         if (outFile.is_open()) {
00150             outFile « SlidingTilesData::slides « std::endl;
00151             outFile.close();
00152         }
00153     }
00154
00155     std::vector<unsigned int> loadScores(SlidingTilesEnums::Difficulty difficulty) {
00156         std::vector<unsigned int> scores;
00157
00158         // Get filename based on difficulty
00159         std::string filename;
00160         switch (difficulty) {
00161         case SlidingTilesEnums::Difficulty::EASY:
00162             filename = "scores_easy.txt";
00163             break;
00164         case SlidingTilesEnums::Difficulty::MEDIUM:
00165             filename = "scores_medium.txt";
00166             break;
00167         case SlidingTilesEnums::Difficulty::HARD:
00168             filename = "scores_hard.txt";
00169             break;
00170         case SlidingTilesEnums::Difficulty::INSANE:
00171             filename = "scores_insane.txt";
00172             break;
00173         }
00174
00175         // Read all scores from file
00176         std::ifstream inFile(filename);
00177         if (inFile.is_open()) {
00178             unsigned int score;
00179             while (inFile » score) {
00180                 scores.push_back(score);
00181             }
```

```
00182             inFile.close();
00183         }
00184
00185         // Sort scores (lowest is best)
00186         std::sort(scores.begin(), scores.end());
00187
00188         // Keep only top 10
00189         if (scores.size() > SlidingTilesFunctionsHelpers::LEADERBOARD_SIZE) {
00190             scores.resize(SlidingTilesFunctionsHelpers::LEADERBOARD_SIZE);
00191         }
00192
00193         return scores;
00194     }
00195 }
```

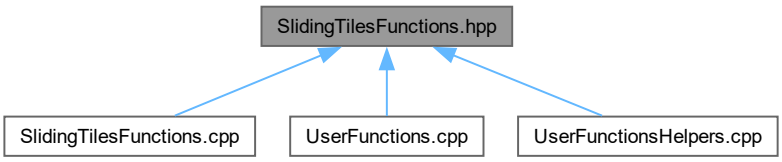
8.15 SlidingTilesFunctions.hpp File Reference

File containing a namespace with declared functions used during a game of Sliding Tiles.

`#include <vector>`
Include dependency graph for SlidingTilesFunctions.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace SlidingTilesEnums

  Namespace containing enum classes for a Sliding Tiles game.
- namespace SlidingTilesFunctions

  Namespace containing the functions used during a game of Sliding Tiles.

Functions

- void SlidingTilesFunctions::startGame (SlidingTilesEnums::Difficulty difficulty)

  Sets up the data required to start a Sliding Tiles game.
- bool SlidingTilesFunctions::slide (SlidingTilesEnums::Direction direction)

  Attempts to slide the empty tile to an adjacent filled tile.
- void SlidingTilesFunctions::shuffle ()

  Shuffles the board.
- bool SlidingTilesFunctions::isBoardOrdered ()

  Checks if all the board's numbers are ordered.
- void SlidingTilesFunctions::writeScore (SlidingTilesEnums::Difficulty difficulty)

  Writes the number of slides to its associated leaderboard.
- std::vector< unsigned int > SlidingTilesFunctions::loadScores (SlidingTilesEnums::Difficulty difficulty)

  Loads the top scores from its associated leaderboard.

8.15.1 Detailed Description

This file contains a namespace with the declaration of functions which are called during a game of Sliding Tiles to manipulate the data of SlidingTilesData.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

SlidingTilesFunctions.cpp

SlidingTilesData

Definition in file SlidingTilesFunctions.hpp.

8.16 SlidingTilesFunctions.hpp

Go to the documentation of this file.

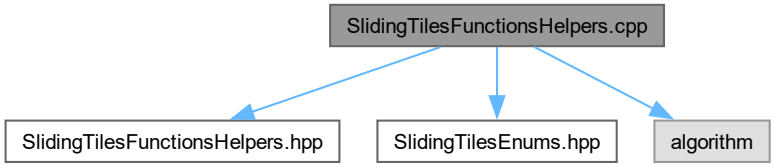
```
00001
00014 #pragma once
00015 #include <vector>
00016
00017 //Forward declarations
00018
00019 namespace SlidingTilesEnums {
00020     enum class Difficulty;
00021     enum class Direction;
00022 }
00023
00031 namespace SlidingTilesFunctions {
00040     void startGame(SlidingTilesEnums::Difficulty difficulty);
00041
00058     bool slide(SlidingTilesEnums::Direction direction);
00059
00078     void shuffle();
00079
00087     bool isBoardOrdered();
00088
00097     void writeScore(SlidingTilesEnums::Difficulty difficulty);
00098
00109     std::vector<unsigned int> loadScores(SlidingTilesEnums::Difficulty difficulty);
00110 }
```

8.17 SlidingTilesFunctionsHelpers.cpp File Reference

File containing the initialization of the SlidingTilesFunctionsHelpers functions.

```
#include "SlidingTilesFunctionsHelpers.hpp"
#include "SlidingTilesEnums.hpp"
#include <algorithm>
```

Include dependency graph for SlidingTilesFunctionsHelpers.cpp:



Namespaces

- namespace SlidingTilesFunctionsHelpers

  Namespace containing helper functions and a constant expression used by `SlidingTilesFunctions`.

Functions

- SlidingTilesEnums::Direction SlidingTilesFunctionsHelpers::getRandomDirection ()

  Generates a random direction.
- SlidingTilesEnums::Direction SlidingTilesFunctionsHelpers::getOpposite (SlidingTilesEnums::Direction dir)

  Converts the direction given into its opposite direction.

8.17.1 Detailed Description

This file contains the initialization of the SlidingTilesFunctionsHelpers helper functions declared in SlidingTilesFunctionsHelpers.hpp.

Author

Rajdeep Chowdhury

Fabio Bustamante Romillo

Date

8.04.2026

See also

SlidingTilesFunctionsHelpers.hpp

SlidingTilesEnums.hpp

SlidingTilesFunctions

Definition in file SlidingTilesFunctionsHelpers.cpp.

8.18 SlidingTilesFunctionsHelpers.cpp

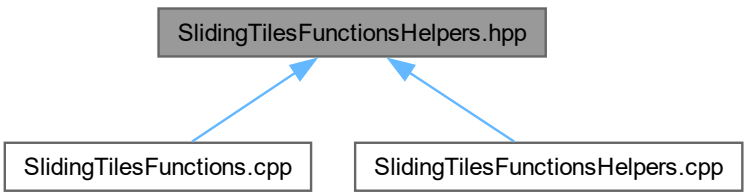
Go to the documentation of this file.

```
00001
00016 #include "SlidingTilesFunctionsHelpers.hpp"
00017 #include "SlidingTilesEnums.hpp"
00018 #include <algorithm>
00019
00020 namespace SlidingTilesFunctionsHelpers {
00021     SlidingTilesEnums::Direction getRandomDirection() {
00022         int r = std::rand() % 4;
00023         return static_cast<SlidingTilesEnums::Direction>(r);
00024     }
00025
00026     SlidingTilesEnums::Direction getOpposite(SlidingTilesEnums::Direction dir) {
00027         switch (dir) {
00028         case SlidingTilesEnums::Direction::UP:    return SlidingTilesEnums::Direction::DOWN;
00029         case SlidingTilesEnums::Direction::DOWN:  return SlidingTilesEnums::Direction::UP;
00030         case SlidingTilesEnums::Direction::LEFT:  return SlidingTilesEnums::Direction::RIGHT;
00031         case SlidingTilesEnums::Direction::RIGHT: return SlidingTilesEnums::Direction::LEFT;
00032         }
00033         return SlidingTilesEnums::Direction::UP; // fallback
00034     }
00035 }
```

8.19 SlidingTilesFunctionsHelpers.hpp File Reference

File containing a namespace of declared helper functions and a constant expression for SlidingTilesFunctions.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace SlidingTilesEnums

  Namespace containing enum classes for a Sliding Tiles game.
- namespace SlidingTilesFunctionsHelpers

  Namespace containing helper functions and a constant expression used by `SlidingTilesFunctions`.

Functions

- SlidingTilesEnums::Direction SlidingTilesFunctionsHelpers::getRandomDirection ()

  Generates a random direction.
- SlidingTilesEnums::Direction SlidingTilesFunctionsHelpers::getOpposite (SlidingTilesEnums::Direction dir)

  Converts the direction given into its opposite direction.

Variables

- constexpr unsigned short SlidingTilesFunctionsHelpers::LEADERBOARD_SIZE = 10

  Constant representing the size of the leaderboard.

8.19.1 Detailed Description

This file contains a namespace of declared helper functions and a `constexpr` integer that the functions of SlidingTilesFunctions use.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

SlidingTilesFunctionsHelpers.cpp

SlidingTilesFunctions

Definition in file SlidingTilesFunctionsHelpers.hpp.

8.20 SlidingTilesFunctionsHelpers.hpp

Go to the documentation of this file.

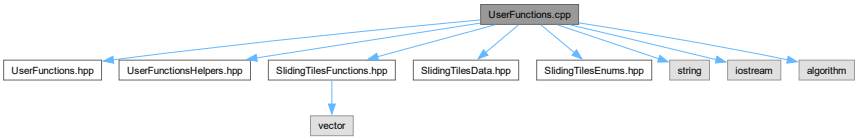
```
00001
00014 #pragma once
00015
00016 //Forward declaration
00017
00018 namespace SlidingTilesEnums {
00019     enum class Direction;
00020 }
00021
00029 namespace SlidingTilesFunctionsHelpers {
00032     constexpr unsigned short LEADERBOARD_SIZE = 10;
00033
00042     SlidingTilesEnums::Direction getRandomDirection();
00043
00054     SlidingTilesEnums::Direction getOpposite(SlidingTilesEnums::Direction dir);
00055 }
```

8.21 UserFunctions.cpp File Reference

File containing the initialization of the UserFunctions functions.

```
#include "UserFunctions.hpp"
#include "UserFunctionsHelpers.hpp"
#include "SlidingTilesFunctions.hpp"
#include "SlidingTilesData.hpp"
#include "SlidingTilesEnums.hpp"
#include <string>
#include <iostream>
#include <algorithm>
```

Include dependency graph for UserFunctions.cpp:



Namespaces

- namespace UserFunctions

  Namespace containing functions that the user interacts with.

Functions

- bool UserFunctions::selectDifficulty ()

  Allows the user to select their Sliding Tiles game difficulty.
- bool UserFunctions::performSlide ()

  Allows the user to perform a slide in a Sliding Tiles game.

8.21.1 Detailed Description

This file contains the initialization of the UserFunctions functions declared in UserFunctions.hpp.

Author

Rajdeep Chowdhury

Date

9.04.2026

See also

UserFunctions.hpp
UserFunctionsHelpers.hpp
SlidingTilesFunctions.hpp
SlidingTilesData.hpp
SlidingTilesEnums.hpp
Main.cpp
UserFunctionsHelpers

Definition in file UserFunctions.cpp.

8.22 UserFunctions.cpp

Go to the documentation of this file.

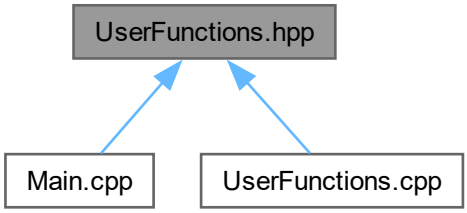
```
00001
00019 #include "UserFunctions.hpp"
00020 #include "UserFunctionsHelpers.hpp"
00021 #include "SlidingTilesFunctions.hpp"
00022 #include "SlidingTilesData.hpp"
00023 #include "SlidingTilesEnums.hpp"
00024 #include <string>
00025 #include <iostream>
00026 #include <algorithm>
00027
00028 namespace UserFunctions {
00029     bool selectDifficulty() {
00030         //Infinute loop to ensure a valid option is picked
00031         while (true) {
00032             //Get and transform user input
00033             std::string input;
00034             std::cout « "SELECT:  [Easy]  [Medium]  [Hard]  [Insane]  [Quit]\n" « std::endl;
00035             std::cin » input;
00036             std::transform(input.begin(), input.end(), input.begin(),
00037                 [](unsigned char character) { return std::tolower(character); });
00038
00039             //Check if it is one of the options;
00040             if (input == "easy" || input == "e") {
00041                 SlidingTilesData::currentDifficulty = SlidingTilesEnums::Difficulty::EASY;
00042                 UserFunctionsHelpers::startGame();
00043                 return true;
00044             }
00045             else if (input == "medium" || input == "m") {
00046                 SlidingTilesData::currentDifficulty = SlidingTilesEnums::Difficulty::MEDIUM;
00047                 UserFunctionsHelpers::startGame();
00048                 return true;
00049             }
00050             else if (input == "hard" || input == "h") {
00051                 SlidingTilesData::currentDifficulty = SlidingTilesEnums::Difficulty::HARD;
00052                 UserFunctionsHelpers::startGame();
00053                 return true;
00054             }
00055             else if (input == "insane" || input == "i") {
00056                 SlidingTilesData::currentDifficulty = SlidingTilesEnums::Difficulty::INSANE;
00057                 UserFunctionsHelpers::startGame();
00058                 return true;
00059             }
00060             else if (input == "quit" || input == "q") {
00061                 return false;
00062             }
00063             else std::cout « "Invalid option" « std::endl;
00064         }
00065     }
00066
00067     bool performSlide() {
00068         //Infinute loop to ensure a valid option is picked
00069         while (true) {
00070             UserFunctionsHelpers::printBoard();
00071
00072             //Get and transform user inpupt
00073             std::string input;
00074             std::cout « "SELECT:  [LEFT/A]  [RIGHT/D]  [UP/W]  [DOWN/S]  [Quit]\n" « std::endl;
00075             std::cin » input;
00076             std::transform(input.begin(), input.end(), input.begin(),
00077                 [](unsigned char character) { return std::tolower(character); });
00078
00079             //Check if it matches one of the options
00080             if (input == "left" || input == "a") {
00081                 UserFunctionsHelpers::trySlide(SlidingTilesEnums::Direction::LEFT);
00082                 return UserFunctionsHelpers::continueGame();
00083             }
00084             else if (input == "right" || input == "d") {
00085                 UserFunctionsHelpers::trySlide(SlidingTilesEnums::Direction::RIGHT);
00086                 return UserFunctionsHelpers::continueGame();
00087             }
00088             else if (input == "up" || input == "w") {
00089                 UserFunctionsHelpers::trySlide(SlidingTilesEnums::Direction::UP);
00090                 return UserFunctionsHelpers::continueGame();
00091             }
00092             else if (input == "down" || input == "s") {
00093                 UserFunctionsHelpers::trySlide(SlidingTilesEnums::Direction::DOWN);
00094                 return UserFunctionsHelpers::continueGame();
00095             }
00096             else if (input == "quit") {
00097                 return false;
00098             }
00099             else std::cout « "Invalid option" « std::endl;
```

```
00100         }
00101     }
00102 }
```

8.23 UserFunctions.hpp File Reference

File containing a namespace with declared functions that the user interacts with.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace UserFunctions

  Namespace containing functions that the user interacts with.

Functions

- bool UserFunctions::selectDifficulty ()

  Allows the user to select their Sliding Tiles game difficulty.
- bool UserFunctions::performSlide ()

  Allows the user to perform a slide in a Sliding Tiles game.

8.23.1 Detailed Description

This file contains a namespace with the declaration of functions that turn user input into output.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

UserFunctions.cpp

Main.cpp

UserFunctionsHelpers

Definition in file UserFunctions.hpp.

8.24 UserFunctions.hpp

Go to the documentation of this file.

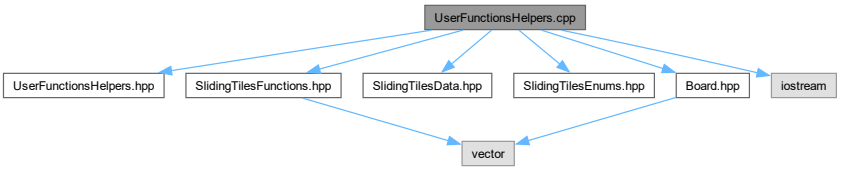
```
00001
00014 #pragma once
00015
00021 namespace UserFunctions {
00029     bool selectDifficulty();
00030
00040     bool performSlide();
00041 }
```

8.25 UserFunctionsHelpers.cpp File Reference

File containing the initialization of the UserFunctionsHelpers functions.

```
#include "UserFunctionsHelpers.hpp"
#include "SlidingTilesFunctions.hpp"
#include "SlidingTilesData.hpp"
#include "SlidingTilesEnums.hpp"
#include "Board.hpp"
#include <iostream>
```

Include dependency graph for UserFunctionsHelpers.cpp:



Namespaces

- namespace UserFunctionsHelpers

  Namespace containing helper functions used by `UserFunctions`.

Functions

- void UserFunctionsHelpers::startGame ()

  Initializes data to start a game of Sliding Tiles.
- void UserFunctionsHelpers::trySlide (SlidingTilesEnums::Direction direction)

  Tries to slide the empty tile in the given direction.
- void UserFunctionsHelpers::printBoard ()

  Print the Sliding Tiles board.
- void UserFunctionsHelpers::printLeaderboard ()

  Print the Sliding Tiles leaderboard for the current difficulty.
- bool UserFunctionsHelpers::continueGame ()

  Determines whether the Sliding Tiles game should continue.

8.25.1 Detailed Description

This file contains the initialization of the UserFunctionsHelpers helper functions declared in UserFunctionsHelpers.hpp.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

UserFunctionsHelpers.hpp

SlidingTilesFunctions.hpp

SlidingTilesData.hpp

SlidingTilesEnums.hpp

Board.hpp

UserFunctions

Definition in file UserFunctionsHelpers.cpp.

8.26 UserFunctionsHelpers.cpp

Go to the documentation of this file.

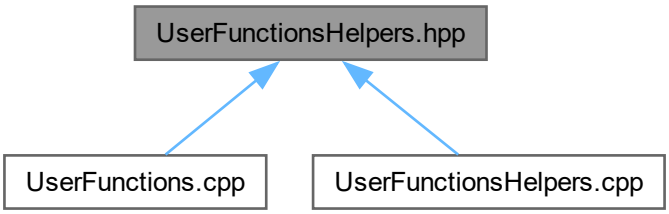
```
00001
00018 #include "UserFunctionsHelpers.hpp"
00019 #include "SlidingTilesFunctions.hpp"
00020 #include "SlidingTilesData.hpp"
00021 #include "SlidingTilesEnums.hpp"
00022 #include "Board.hpp"
00023 #include <iostream>
00024
00025 namespace UserFunctionsHelpers {
00026     void startGame() {
00027         printLeaderboard();
00028         SlidingTilesFunctions::startGame(SlidingTilesData::currentDifficulty);
00029         SlidingTilesFunctions::shuffle();
00030     }
00031
00032     void trySlide(SlidingTilesEnums::Direction direction) {
00033         //Perform diffefrent results depending on if the slide succeeds or not
00034         bool success = SlidingTilesFunctions::slide(direction);
00035         if (success) SlidingTilesData::slides++;
00036         else std::cout « "No tile found in that direction." « std::endl;
00037     }
00038
00039     void printBoard() {
00040         //Use a nested for loop
00041         for (int i = 0; i < SlidingTilesData::boardSize; i++) {
00042             if (!SlidingTilesData::board.canAccess(i)) break; //For safety
00043             for (size_t tile : SlidingTilesData::board.access(i)) {
00044                 if (tile == SlidingTilesData::boardSize * SlidingTilesData::boardSize) std::cout « "[
    ]";
00045                 else if (tile <= 9) std::cout « "[ " « tile « ']';
00046                 else std::cout « '[' « tile « ']';
00047             }
00048             std::cout « '\n';
00049         }
00050     }
00051
00052     void printLeaderboard() {
00053         //Load top scores then print them
```

```
00054         std::vector<unsigned int> scores =
      SlidingTilesFunctions::loadScores(SlidingTilesData::currentDifficulty);
00055         std::cout « "\nLEADERBOARD\n";
00056         for (int i = 0; i < scores.size(); i++) {
00057             std::cout « i + 1 « ". " « scores.at(i) « '\n';
00058         }
00059         std::cout « '\n';
00060     }
00061
00062     bool continueGame() {
00063         using namespace SlidingTilesData;
00064         //The last move of the sliding tiles game is always making the empty tile go to the bottom
      right corner
00065         if (board.access(boardSize - 1, boardSize - 1) != boardSize * boardSize) return true;
00066         //Then check if the board is ordered to trigger a win
00067         else if (SlidingTilesFunctions::isBoardOrdered()) {
00068             SlidingTilesFunctions::writeScore(currentDifficulty);
00069             printBoard();
00070             std::cout « "Winner! \n";
00071             printLeaderboard();
00072             return false;
00073         }
00074         else return true;
00075     }
00076
00077 }
```

8.27 UserFunctionsHelpers.hpp File Reference

File containing a namespace of declared helper functions and a constant expression for UserFunctions.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace SlidingTilesEnums

  Namespace containing enum classes for a Sliding Tiles game.
- namespace UserFunctionsHelpers

  Namespace containing helper functions used by `UserFunctions`.

Functions

- void UserFunctionsHelpers::startGame ()

  Initializes data to start a game of Sliding Tiles.
- void UserFunctionsHelpers::trySlide (SlidingTilesEnums::Direction direction)

Tries to slide the empty tile in the given direction.

- void UserFunctionsHelpers::printBoard ()

  Print the Sliding Tiles board.

- void UserFunctionsHelpers::printLeaderboard ()

  Print the Sliding Tiles leaderboard for the current difficulty.

- bool UserFunctionsHelpers::continueGame ()

  Determines whether the Sliding Tiles game should continue.

8.27.1 Detailed Description

This file contains a namespace of declared helper functions that the functions of UserFunctions use.

Author

Rajdeep Chowdhury

Date

8.04.2026

See also

UserFunctionsHelpers.cpp

UserFunctions

Definition in file UserFunctionsHelpers.hpp.

8.28 UserFunctionsHelpers.hpp

Go to the documentation of this file.

```
00001
00013 #pragma once
00014
00015 //Forward declaration
00016
00017 namespace SlidingTilesEnums {
00018     enum class Direction;
00019 }
00020
00027 namespace UserFunctionsHelpers {
00033     void startGame();
00034
00040     void trySlide(SlidingTilesEnums::Direction direction);
00041
00046     void printBoard();
00047
00056     void printLeaderboard();
00057
00065     bool continueGame();
00066 }
```